COMPILADORES

Exercícios práticos — Aulas 6 a 9

Enunciados para estudo e implementação

Lista de exercícios práticos

Conteúdos: aulas 6 a 9 — especificação, gramáticas, análise sintática, AST, descida recursiva, transformações e FIRST/FOLLOW.

Orientação geral: apresente as decisões adotadas, justifique transformações e diferencie árvore de derivação de AST. Quando solicitado, use $ como marcador de fim de entrada.

Exercício 1 — Da especificação léxica à gramática sintática

Aulas 6 e 7 — tokens, gramática e descida recursiva

1. Considere a expressão aritmética com identificadores, inteiros e os operadores + e *. Produza uma gramática livre de contexto que represente precedência usual e associe cada terminal ao token produzido pelo scanner.

2. Elabore uma estratégia de análise descendente para a gramática. Identifique qualquer recursão à esquerda e apresente sua transformação.

3. Para a entrada id + num * id, registre a sequência de decisões do analisador e a AST resultante.

Caso de teste

```
id + num * id
```

Resultado a ser verificado

AST esperada: +(id, *(num, id))

Exercício 2 — Eliminação de recursão à esquerda

Aula 7 — transformação de gramáticas

1. Transforme a gramática E → E + T | T; T → T * F | F; F → (E) | id para uma forma adequada à análise descendente.

2. Explique por que a gramática original não pode ser implementada diretamente por descida recursiva.

3. Compare as árvores de derivação conceitual antes e depois da transformação e mostre que a precedência foi preservada.

Caso de teste

```
id + id * id
```

Resultado a ser verificado

Derivação compatível com +(id, *(id, id))

Exercício 3 — Construção de AST para comandos

Aulas 6 e 7 — sintaxe abstrata e parsing

1. Defina uma AST mínima para atribuição, literal inteiro, variável e comando condicional.

2. Para o código fornecido, construa a AST sem incluir pontuação desnecessária, como parênteses e ponto e vírgula.

3. Indique quais funções do parser poderiam construir cada nó.

Caso de teste

```
x = 4; if (x) y = x + 1;
```

Resultado a ser verificado

Assign(x, 4); If(x, Assign(y, +(x, 1)), Empty)

Exercício 4 — Escolha de produção com FIRST e FOLLOW

Aula 9 — análise descendente preditiva

1. Calcule FIRST e FOLLOW para a gramática transformada do Exercício 2.

2. Preencha as entradas não vazias da tabela preditiva M[A,a].

3. Simule a análise da entrada id + id * id $, exibindo pilha, entrada restante e ação em cada passo.

Caso de teste

```
id + id * id $
```

Resultado a ser verificado

A cadeia é aceita; a tabela não apresenta conflitos para os terminais relevantes.

Exercício 5 — Diagnóstico de estratégia sintática

Aulas 8 e 9 — comparação de estratégias

1. Analise a gramática S → a S b | a b e determine se ela é adequada para uma implementação descendente direta.

2. Proponha uma alternativa de estratégia: transformação da gramática, parser ascendente ou outra abordagem justificada.

3. Para a entrada a a b b, apresente a derivação ou sequência de reduções escolhida e a árvore/AST correspondente.

Caso de teste

```
a a b b
```

Resultado a ser verificado

A cadeia é aceita pela linguagem $\{a^n b^n \mid n \geq 1\}$; AST/árvore: S(a, S(a,b), b)

Critérios de entrega

- gramática legível e consistente
- transformações justificadas
- AST com nós e argumentos explícitos
- rastreamento de análise suficientemente detalhado
- distinção entre tokens, terminais e não terminais